

PRJ – Implementing a Processor on FPGA

Figure 1 shows a digital system that contains a number of 9-bit registers, a multiplexer, an adder/subtractor unit, and a control unit (finite state machine). Data is input to this system via the 9-bit DIN input. This data can be loaded through the 9-bit wide multiplexer into the various registers, such as R0, ..., R7 and A. The multiplexer also allows data to be transferred from one register to another. The multiplexer's output wires are called a bus in the figure because this term is often used for wiring that allows data to be transferred from one location in a system to another.

Addition or subtraction is performed by using the multiplexer to first place one 9-bit number onto the bus wires and loading this number into register A. Once this is done, a second 9-bit number is placed onto the bus, the adder/subtractor unit performs the required operation, and the result is loaded into register G. The data in G can then be transferred to one of the other registers as required.

The system can perform different operations in each clock cycle, as governed by the **control unit**. This unit determines when particular data is placed onto the bus wires and it controls which of the registers is to be loaded with this data. For example, if the control unit asserts the signals $R0_{out}$ and A_{in} , then the multiplexer will place the contents of register R0 onto the bus and this data will be loaded by the next active clock edge into register A.

A system like this is often called a **processor**. It executes operations specified in the form of instructions. Table 1 lists the instructions that the processor has to support for this exercise. The left column shows the name of an instruction and its operand. The meaning of the syntax $RX \leftarrow [RY]$ is that the contents of register RY are loaded into register RX. The **mv** (move) instruction allows data to be copied from one register to another. For the **mvi** (move immediate) instruction the expression $RX \leftarrow D$ indicates that the 9-bit constant D is loaded into register RX.

this exercise. Hence IR has to be connected to the nine bits of the DIN input, as indicated in Figure 1. For the **mvi** instruction the **YYY** field has no meaning, and the immediate data **#D** has to be supplied on the 9-bit DIN input after the **mvi** instruction word is stored into IR.

Some instructions, such as an addition or subtraction, take more than one clock cycle to complete, because multiple transfers have to be performed across the bus. The finite state machine in the control unit “steps through” such instructions, asserting the control signals needed in successive clock cycles until the instruction has completed. The processor starts executing the instruction on the DIN input when the *Run* signal is asserted and the processor asserts the *Done* output when the instruction is finished. Table 2 indicates the control signals that can be asserted in each time step to implement the instructions in Table 1. Note that the only control signal asserted in time step 0 is IR_{in} , so this time step is not shown in the table.

	T_1	T_2	T_3
(mv): I_0	$RY_{out}, RX_{in},$ <i>Done</i>		
(mvi): I_1	$DIN_{out}, RX_{in},$ <i>Done</i>		
(add): I_2	RX_{out}, A_{in}	RY_{out}, G_{in}	$G_{out}, RX_{in},$ <i>Done</i>
(sub): I_3	RX_{out}, A_{in}	$RY_{out}, G_{in},$ <i>AddSub</i>	$G_{out}, RX_{in},$ <i>Done</i>

Table 2. Control signals asserted in each instruction/time step.

- I. Design and implement the processor shown in Figure 1 using VHDL code as follows:
 - a. Describe the FSM allowing to implement the behavior of this processor.
 - b. Create a new Vivado project for this exercise.
 - c. Generate the required VHDL files.
 - d. Use functional simulation to test the circuit.
 - e. Compile the circuit and download it into the FPGA chip. Test the functionality of your design.

- II. In this part you are to design the circuit depicted in Figure 3, in which a memory module and counter are connected to the processor from Part I. The counter is used to read the contents of successive addresses in the memory, and this data is provided to the processor as a stream of instructions. To simplify the design and testing of this circuit we have used separate clock signals, *PClock* and *MClock*, for the processor and memory.

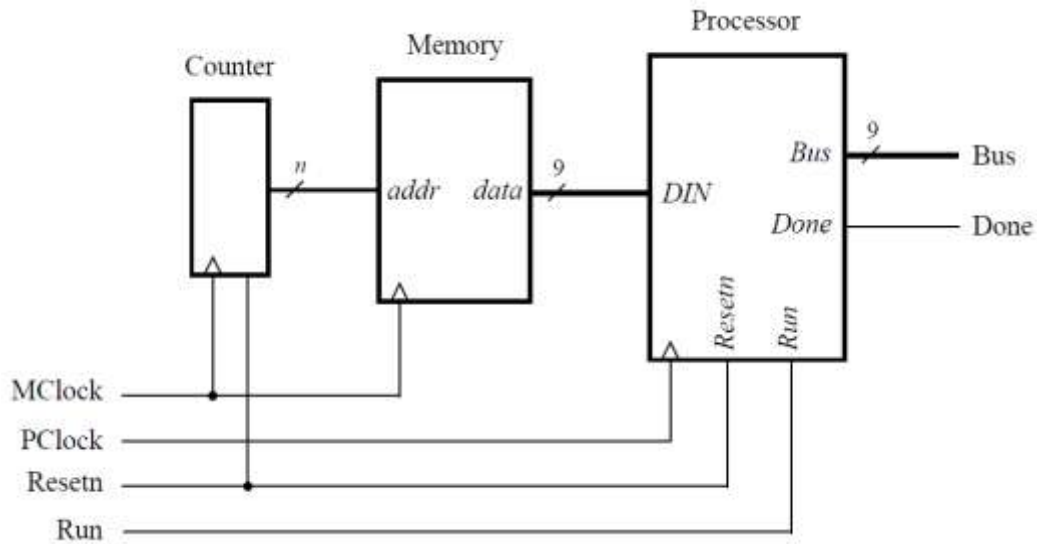


Figure 2. Connecting the processor to a memory and counter.

- Create a new Vivado project which will be used to test your circuit.
- Generate a top-level VHDL file that instantiates the processor, memory, and counter. To place processor instructions into the memory, you need to specify initial values that should be stored in the memory once your circuit has been programmed into the FPGA chip.
- Use functional simulation to test the circuit. Ensure that data is read properly out of the ROM and executed by the processor.
- Compile the circuit and download it into the FPGA chip. Test the functionality of your design.

- III. In this part you will extend the capability of the processor so that the external counter is no longer needed, and so that the processor has the ability to perform read and write operations using memory or other devices. You will add three new types of instructions to the processor, as displayed in Table 3. The **ld** (load) instruction loads data into register RX from the external memory address specified in register RY. The **st** (store) instruction stores the data contained in register RX into the memory address found in RY. Finally, the instruction **mvnz** (move if not zero) allows a **mv** operation to be executed only under a certain condition; the condition is that the current contents of register G are not equal to 0.

Operation	Function performed
ld $R_x, [R_y]$	$R_x \leftarrow [[R_y]]$
st $R_x, [R_y]$	$[R_y] \leftarrow [R_x]$
mvnz R_x, R_y	if $G \neq 0$, $R_x \leftarrow [R_y]$

Table 3. New instructions performed in the processor.

A schematic of the enhanced processor is given in Figure 3. In this figure, registers R0 to R6 are the same as in Figure 1, but register R7 has been changed to a counter. This counter is used to provide the addresses in the memory from which the processor's instructions are read. We will refer to R7 as the processor's *program counter* (PC), because this terminology is common for real processors available in the industry. When the processor is reset, PC is set to address 0. At the start of each instruction (in time step 0) the contents of PC are used as an address to read an instruction from the memory. The instruction is stored in IR and the PC is automatically incremented to point to the next instruction (in the case of **movi** the PC provides the address of the immediate data and is then incremented again).

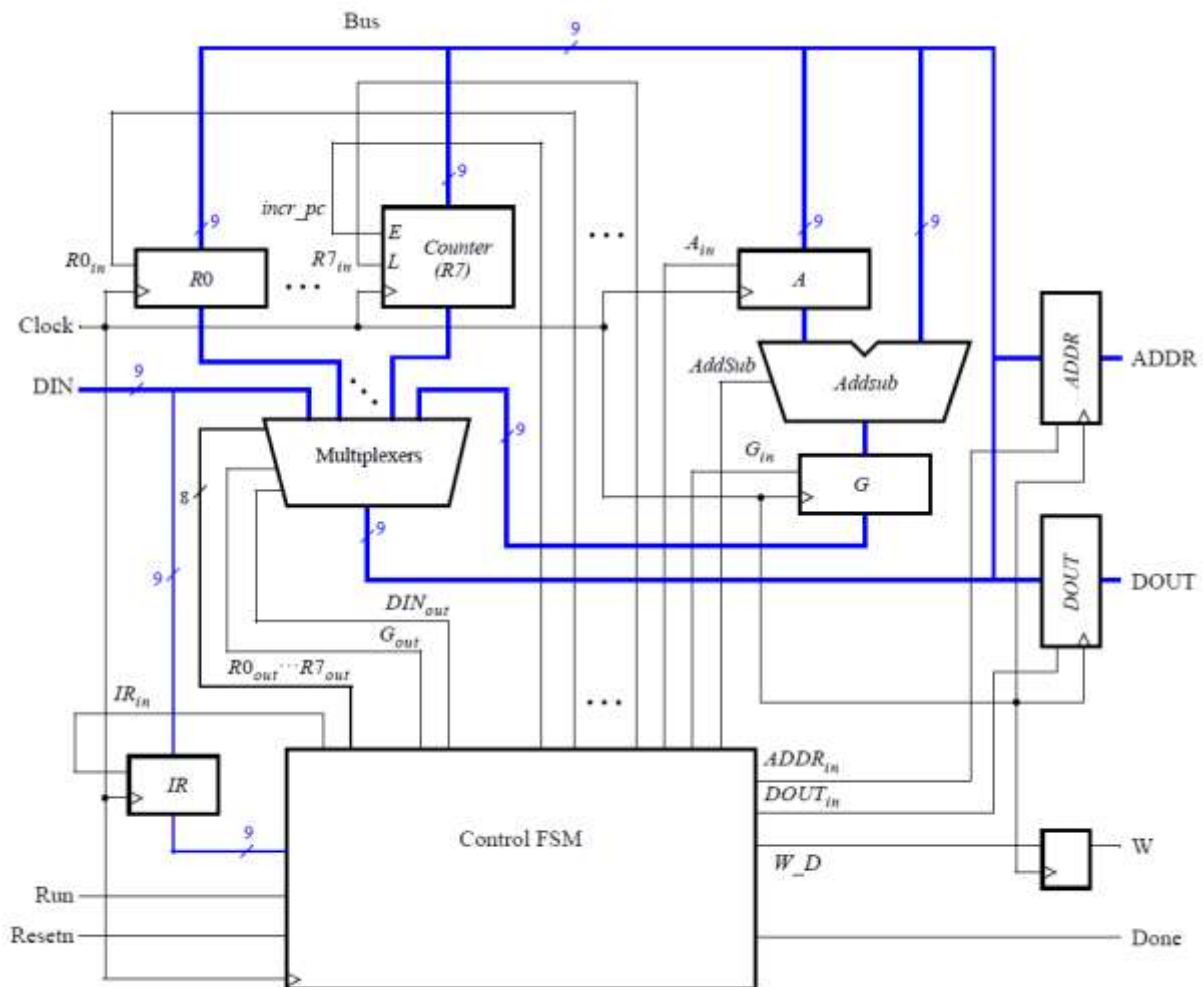


Figure 3. An enhanced version of the processor.

The processor's control unit increments PC by using the *incr_PC* signal, which is just an enable on this counter. It is also possible to directly load an address into PC (R7) by having the processor execute a **mv** or **mvi** instruction in which the destination register is specified as R7. In this case the control unit uses the signal *R7_{in}* to perform a parallel load of the counter. In this way, the processor can execute instructions at any address in memory, as opposed to only being able to execute instructions that are stored in successive addresses. Similarly, the current contents of PC can be copied into another register by using a **mv** instruction. An example of code that uses the PC register to implement a loop is shown below, where the text after the % on each line is just a comment. The instruction **mv R5, R7** places into R5 the address in memory of the instruction **sub R4, R2**. Then, the instruction **mvnz R7, R5** causes the **sub** instruction to be executed repeatedly until R4 becomes 0. This type of loop could be used in a larger program as a way of creating a delay.

```

mvi    R2, #1
mvi    R4, #10000000    % binary delay value
mv     R5, R7           % save address of next instruction
sub    R4, R2           % decrement delay count
mvnz   R7, R5           % continue subtracting until delay count gets to 0

```

Figure 3 shows two registers in the processor that are used for data transfers. The ADDR register is used to send addresses to an external device, such as a memory module, and the

DOUT register is used by the processor to provide data that can be stored outside the processor. One use of the ADDR register is for reading, or *fetching*, instructions from memory; when the processor wants to fetch an instruction, the contents of PC (R7) are transferred across the bus and loaded into ADDR. This address is provided to memory. In addition to fetching instructions, the processor can read data at any address by using the ADDR register. Both data and instructions are read into the processor on the DIN input port. The processor can write data for storage at an external address by placing this address into the ADDR register, placing the data to be stored into its DOUT register, and asserting the output of the W (write) flip-flop to 1.

Figure 4 illustrates how the enhanced processor is connected to memory and other devices. The memory unit in the figure supports both read and write operations and therefore has both address and data inputs, as well as a write enable input. The memory also has a clock input, because the address, data, and write enable inputs must be loaded into the memory on an active clock edge. This type of memory unit is usually called a *synchronous random access memory* (synchronous RAM). Figure 4 also includes a 9-bit register that can be used to store data from the processor; this register might be connected to a set of LEDs to allow display of data on the DE1 board. To allow the processor to select either the memory unit or register when performing a write operation, the circuit includes some logic gates that perform *address decoding*: if the upper address lines are $A_8A_7 = 00$, then the memory module will be written at the address given on the lower address lines. Figure 4 shows n lower address lines connected to the memory; for this exercise a memory with 128 words is probably sufficient, which implies that $n = 7$ and the memory address port is driven by $A_6 \dots A_0$. For addresses in which $A_8A_7 = 01$, the data written by the processor is loaded into the register whose outputs are called LEDs in Figure 4.

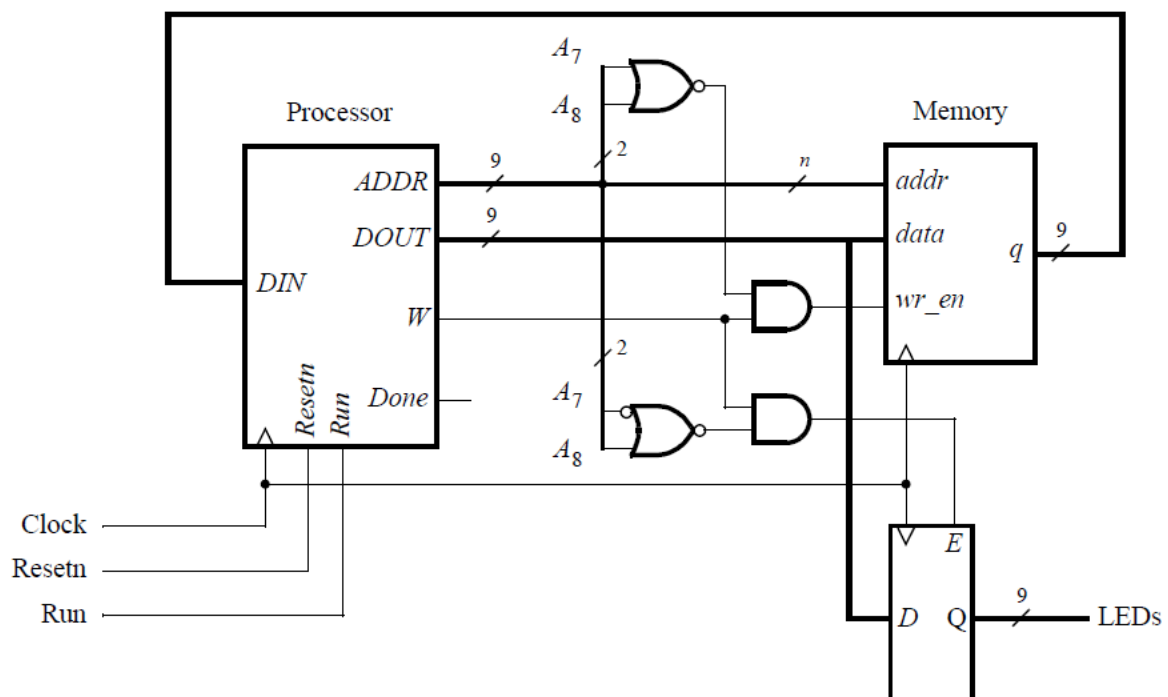


Figure 4. Connecting the enhanced processor to a memory and output register.

- a. Create a new Vivado project for the enhanced version of the processor.
- b. Write VHDL code for the processor and test your circuit by using functional simulation: apply instructions to the DIN port and observe the internal processor signals as the instructions are executed. Pay careful attention to the timing of signals between your processor and external memory; account for the fact that the memory has registered input ports, as we discussed for Figure 4.
- c. Create another Vivado project that instantiates the processor, memory module, and register shown in Figure 4.
- d. Use functional simulation to test the circuit. Ensure that data is read properly from the RAM and executed by the processor.
- e. Compile the circuit and download it into the FPGA chip. Test the functionality of your design.